ASSIGNMENT 3

DataScience - Developing a Provenance Monitor for Genomic Data with R Shiny

La Salle URL
Carolina López Larraz

Table of contents

JSON creation and Mongo DB *2*

Shiny App *3*

Introduction 3

Packages 3

Data Pipeline: from MongoDB connection to KPIs 4

Data Pipeline: UI 5

Images of the App *7*

References *11*

JSON creation and Mongo DB

The creation of the dataset constitutes a fundamental step in the system, as it provides the structured input required for the dashboard analysis. According to the assignment description, the provenance information must follow the JSON-LD format based on the PROV-O standard, where each record represents an Activity including metadata such as execution time, software agents, and generated entities.

To simulate a realistic environment, I developed a Python script that generates 100 provenance JSON records. Each record is assigned a unique identifier using the uuid library, guaranteeing traceability across executions.

A key aspect of the generation process is the simulation of execution nodes with different performance profiles. Four nodes are defined ("atlas-qc-01", "orion-align-02", "kepler-variant-03", and "nova-storage-04"), each associated with a speed category and failure probability. This allows the dataset to reflect heterogeneous system behavior, where some nodes are faster and more reliable, while others are slower and more error-prone. Execution duration is calculated as a function of input data size, introducing a realistic relationship between workload and processing time.

The input data is simulated dynamically, generating between 1 and 4 FASTQ files with randomized sizes. This results in different total sizes per execution, enabling analysis of efficiency and throughput. Timestamps are generated using controlled randomness, ensuring executions are distributed over a realistic time window.

Another important component is the simulation of data integrity checks. SHA256 and Seqfu validations are modeled probabilistically based on node failure rates, allowing the dashboard to detect failures and compute system health.

While the generated JSON structure follows the original example, several modifications improve analytical usability.

- First, a new field status was added inside each generated entity, enabling direct classification of executions as "OK" or "FAIL".
- Second, the used relationship was incorporated to represent input entities and improve traceability.
- Third, the wasInformedBy relationship was optionally added to simulate dependencies between activities.
- Additionally, numeric values such as totalSizeBytes and fileCount are stored as strings, and user information preserves the original labeling format.

Shiny App

Introduction

The objective of this project is to design and implement an interactive dashboard to monitor both the performance and the provenance of a computational pipeline. In many data processing environments, it is common to evaluate executions only based on numerical metrics such as execution time or processed data size. However, this approach is incomplete, since it does not provide information about how different executions are connected, which agents were involved, or what data was generated during the process.

For this reason, in this project, I developed a dashboard that integrates two complementary perspectives: on one hand, performance monitoring through metrics and visualizations, and on the other hand, provenance tracking through relational structures extracted from the data. The goal is to allow the user not only to detect inefficiencies or failures, but also to understand the workflow behind each execution.

The application is implemented using reactive programming, meaning that all outputs automatically update when the user interacts with the interface. This allows dynamic exploration of the data, starting from a global overview (KPIs and aggregated plots) and progressively moving towards a detailed analysis of individual executions, including their associated agents, generated entities, and raw JSON structure.

Packages

The implementation of the dashboard is based on a set of R packages, each chosen to address a specific requirement of the system. The selection of these tools is directly related to the nature of the data and the type of analysis that needs to be performed.

- To build the application interface and manage interactivity, I used **Shiny**. This framework provides functions such as **fluidPage(), sidebarLayout(), renderText()** and **renderPlotly(),** which allow defining both the structure of the interface and the dynamic behavior of the outputs.
- For data storage, the system uses **MongoDB**, accessed from R through the mongolite package. The connection is established using the function **mongo()**, and data is retrieved using **find().** MongoDB is particularly appropriate in this case because the provenance data is stored as **JSON files**. A relational database would require flattening these structures, which would difficult both storage and querying.

- Data transformation is performed using **dplyr**, mainly through functions such as **mutate(), filter(), group_by()** and **summarise().** These functions allow transforming and aggregating the data in a clear and structured way. In addition, I used **purrr**, specifically the function **map_chr()**, to extract values from nested fields such as

generated or wasAssociatedWith. This is necessary because many attributes are stored as lists rather than simple columns.

- For visualization, I used **plotly**, with functions such as **plot_ly()** and **layout().** This package was selected because it provides interactive plots, allowing the user to hover over points, inspect values, and explore patterns dynamically. For tabular data, I used **DT**, specifically the function **datatable(),** which enables row selection and interactive exploration.
- Finally, for the provenance representation, I used **visNetwork**, with functions such as **visNetwork(), visNodes()** and **visEdges().** This allows constructing a graph where nodes represent activities, agents, and entities, and edges represent relationships between them. Additionally, I used **jsonlite**, with the function **toJSON(),** to display the raw JSON of each execution in a readable format.

Data Pipeline: from MongoDB connection to KPIs

The first step of the system consists of establishing a connection to the MongoDB database and retrieving execution logs. This is implemented using the **mongo()** function from the mongolite package, specifying the collection ("logs") and connection URL. Once connected, the **find()** function retrieves all documents, automatically converted into a data frame–like structure while preserving nested JSON fields. This is essential because provenance data is hierarchical, containing elements such as generated entities, used inputs, and associated agents.

A key design decision is not to preprocess or aggregate data in the database. Instead, all transformations are performed in R, providing flexibility. For example, execution duration or failure definitions can be modified without changing the database schema. This also simplifies debugging, since the original raw data is always available.

However, MongoDB data introduces complexity due to nested lists. Fields such as generated, used, and wasAssociatedWith contain multiple elements with their own attributes and cannot be directly analyzed. For instance, the generated field may include entries like "Verificació SHA256" and "Verificació Seqfu", each with its own status. Therefore, these values must be explicitly extracted and flattened.

To handle this, I use **mutate()** from dplyr to create and modify variables. I first rename fields such as label to sample and executionNode to node to improve readability. Then, I convert timestamps (startTime, endTime) using **as.POSIXct()**, enabling time calculations. Execution duration is computed with **difftime()** and converted to numeric. For example, a process from 09:02:00 to 09:08:39 results in approximately 399 seconds.

Nested JSON extraction is performed using **map_chr()** from purrr. For example, .x$status[1] and .x$status[2] extract SHA256 and Seqfu statuses, assuming a consistent structure. Similarly, totalSizeBytes is extracted, converted to numeric, and transformed into GB by dividing by 1e9 (for instance, 4,550,480,364 bytes is approximately 4.55 GB).

User extraction is handled from wasAssociatedWith by filtering elements where @type == "Person". This isolates human users (for example, “lab_tech”), and strings are cleaned using **sub()** to remove prefixes.

A global execution status is defined using ifelse(): a run is “OK” only if both subprocesses are successful; otherwise, it is “FAIL”.

Additional performance metrics are computed: efficiency (seconds per GB) and throughput (GB per second). For example, processing 2 GB in 200 seconds yields 100 s/GB and 0.01 GB/s. These metrics allow comparisons independent of dataset size.

Executions are categorized (“small”, “medium”, “large”) using case_when(), facilitating grouped analysis.

Finally, several KPIs summarize system performance: total runs (nrow()), failure rate (mean(df()$status == "FAIL")), average duration (mean()), and total processed data (sum()). Correlation between data size and execution time is calculated with cor(); for example, a value of 0.79 indicates a strong positive relationship.

An alert mechanism based on failure rate displays “System OK” or a warning message depending on a defined threshold, providing a quick indication of system health.

Data Pipeline: UI

The second component of the system corresponds to the user interface (UI), implemented using the shiny package. The interface is designed following a dashboard structure, allowing intuitive exploration of both performance metrics and provenance information.

The layout is organized using **fluidPage()** and **sidebarLayout()**, which divide the application into two main sections: a sidebar for filtering and a main panel for visualization and analysis. This separation is essential to maintain clarity while enabling real-time interaction.

The sidebar panel provides filtering capabilities that dynamically control the dataset displayed in the dashboard. Filters are implemented using **checkboxGroupButtons()** from the shinyWidgets package, which offers a more modern and user-friendly alternative to standard checkboxes. Users can filter executions by:

- execution node,
- user,
- and execution status (OK or FAIL).

These filters are directly connected to a reactive dataset (**df()**), ensuring that all visualizations and KPIs update automatically whenever the user modifies the selection. This reactive behavior is a core feature of Shiny and allows seamless data exploration without reloading the application.

The main panel contains multiple layers of information, starting with KPI cards. These are implemented using **fluidRow()** and **column()** elements combined with styled div() containers. Each card highlights a key metric, such as total runs, failure rate, average

duration, total processed data, dependencies, and correlation. The use of color-coded backgrounds enhances readability and allows users to quickly interpret system performance at a glance.

Below the KPIs, the interface includes a tab-based navigation system created with **tabsetPanel().** This structure organizes the analysis into distinct sections:

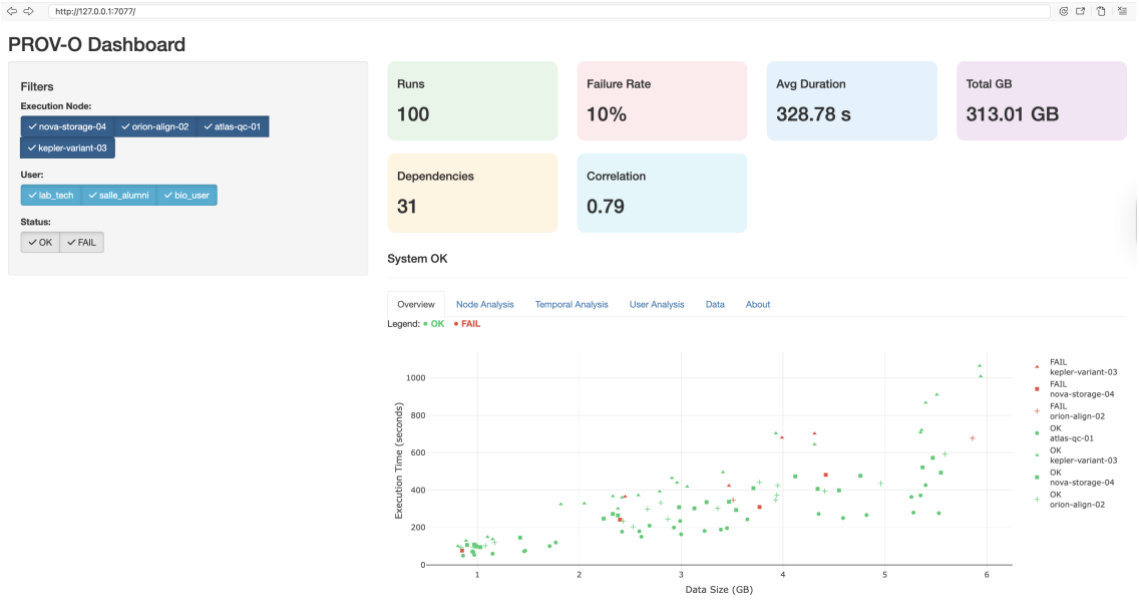
- The Overview tab provides general insights into system behavior. It includes a scatter plot showing the relationship between data size and execution time, a bar plot for execution status distribution, and a size category distribution. These plots are implemented using plotly, enabling interactivity such as zooming, hovering, and dynamic highlighting.
- The Node Analysis tab focuses on performance differences across execution nodes. It includes boxplots for execution time distribution, as well as bar charts for average duration and efficiency. This allows identification of performance variability and potential bottlenecks.
- The Temporal Analysis tab visualizes execution activity over time, grouping runs by hourly intervals. This helps detect workload patterns and peak usage periods.
- The User Analysis tab displays the number of executions per user, supporting usage tracking and system auditing.
- A particularly important component is the Data tab, which enables detailed exploration of individual executions. A **datatable()** from the DT package displays the filtered dataset, allowing row selection. When a row is selected, additional UI elements are dynamically rendered:
    - an activity card summarizing execution details,
    - a list of agents involved,
    - and the generated entities, including their status.

  In addition, the raw provenance data can be inspected using a collapsible JSON viewer, implemented with **toJSON()** from the jsonlite package. This ensures full transparency and traceability of the underlying data.

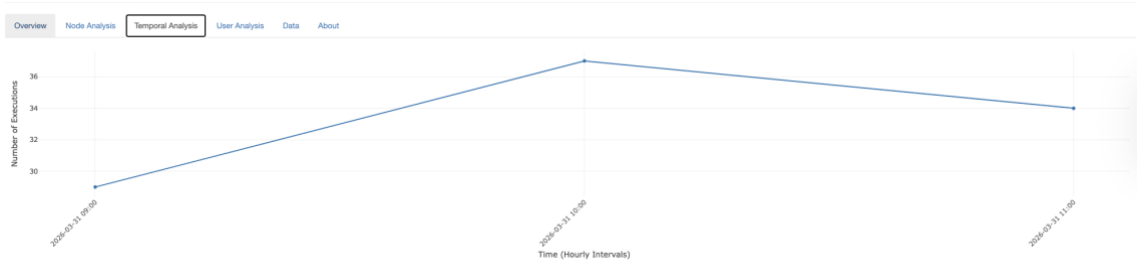
  The most advanced feature of the UI is the provenance graph, implemented using the visNetwork package. This graph represents the execution as a network of nodes and edges following the W3C PROV model. Nodes correspond to activities, agents, and entities, while edges represent relationships such as wasAssociatedWith, used, generated, and wasInformedBy. The graph uses a hierarchical layout, directional edges, and color-coded groups to improve interpretability. This visualization provides an intuitive representation of workflow dependencies and data lineage.

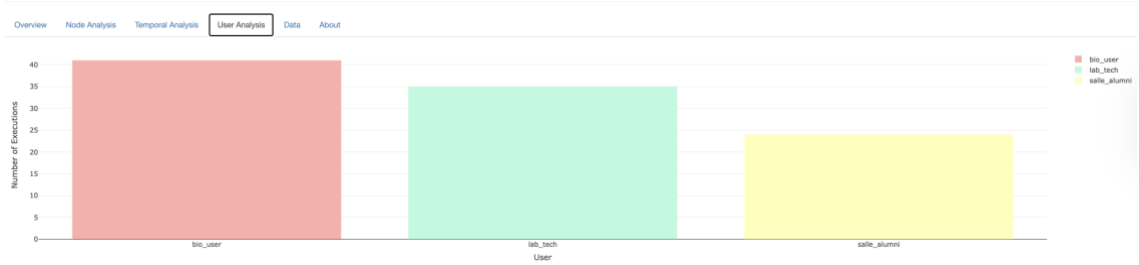
Finally, an alert mechanism is integrated into the interface, displaying system health status based on the failure rate. This provides immediate feedback to the user and supports monitoring tasks.

Images of the App



These images show the main dashboard of the app, where you can analyze how the pipeline executions behave. At the top, there are some summary boxes with key information like number of runs, failure rate, average duration, and total data processed. There are also filters to select node, user, or status. The scatter plots help visualize how execution time changes with data size. The bottom part focuses more on the bar charts, which summarize the results in a simpler way. One chart compares successful and failed executions, showing that most runs were completed correctly. The other chart groups executions by data size (small, medium, large), so you can quickly see which type of data is processed more often. Overall, this section helps to understand the distribution of results at a glance.





These images show different analysis sections of the app that help understand how the system behaves in more detail. In the first part, the boxplots represent how execution times vary for each node, so you can see not only the average but also how consistent or spread out the results are. The bar charts below make it easier to compare average execution time and efficiency between nodes, giving a clearer idea of which ones perform better.

The temporal analysis shows how the number of executions changes over time, helping to detect peaks of activity. Finally, the user analysis compares how many executions each user has performed, which helps to understand usage patterns and who is using the system the most.

Overview | Node Analysis | Temporal Analysis | User Analysis | Data | About

Show 10 entries Search:

	@context	@id	@type	label	startTime	endTime	executionNode	sourceDirectory	destinationDirectory	wasAssociatedWith	generated	used	wasInformedBy	sample	node	duration	sha_st
1	http://www.w3.org/ns/prov#	urn:uuid:255fa29a-558f-4ae8-a84c-36b02b9c00b3	Activity	Processament complet de sample_1	2026-03-31T09:02:00Z	2026-03-31T09:08:39Z	nova-storage-04	/data/input/	/data/output/sample_1	[object Object],[object Object],[object Object],[object Object]	[object Object],[object Object],[object Object],[object Object]	[object Object]		Processament complet de sample_1	nova-storage-04	399	OK
2	http://www.w3.org/ns/prov#	urn:uuid:f3a3ba96-0241-44e3-8f3a-0ba9409a115d	Activity	Processament complet de sample_2	2026-03-31T10:25:00Z	2026-03-31T10:33:14Z	nova-storage-04	/data/input/	/data/output/sample_2	[object Object],[object Object],[object Object],[object Object]	[object Object],[object Object],[object Object],[object Object]	[object Object]		Processament complet de sample_2	nova-storage-04	494	OK
3	http://www.w3.org/ns/prov#	urn:uuid:2fdb0865-1315-4a9a-af95-46447124db05	Activity	Processament complet de sample_3	2026-03-31T11:09:00Z	2026-03-31T11:13:59Z	orion-align-02	/data/input/	/data/output/sample_3	[object Object],[object Object],[object Object],[object Object]	[object Object],[object Object],[object Object],[object Object]	[object Object]	urn:uuid:f3a3ba96-0241-44e3-8f3a-0ba9409a115d	Processament complet de sample_3	orion-align-02	299	OK
4	http://www.w3.org/ns/prov#	urn:uuid:74986de5-7978-47f4-9aba-72f498837f2e	Activity	Processament complet de sample_4	2026-03-31T09:42:00Z	2026-03-31T09:44:31Z	atlas-qc-01	/data/input/	/data/output/sample_4	[object Object],[object Object],[object Object],[object Object]	[object Object],[object Object],[object Object],[object Object]	[object Object]	urn:uuid:2fdb0865-1315-4a9a-af95-46447124db05	Processament complet de sample_4	atlas-qc-01	151	OK
5	http://www.w3.org/ns/prov#	urn:uuid:dfe5dfae-2b02-49c3-8db4-508b2d464adb	Activity	Processament complet de sample_5	2026-03-31T10:33:00Z	2026-03-31T10:39:14Z	orion-align-02	/data/input/	/data/output/sample_5	[object Object],[object Object],[object Object],[object Object]	[object Object],[object Object],[object Object],[object Object]	[object Object]		Processament complet de sample_5	orion-align-02	374	OK
6	http://www.w3.org/ns/prov#	urn:uuid:261b9cb2-8beb-401e-bd31-e01290d3f99b	Activity	Processament complet de sample_6	2026-03-31T11:54:00Z	2026-03-31T11:55:47Z	nova-storage-04	/data/input/	/data/output/sample_6	[object Object],[object Object],[object Object],[object Object]	[object Object],[object Object],[object Object],[object Object]	[object Object]		Processament complet de sample_6	nova-storage-04	107	OK
7	http://www.w3.org/ns/prov#	urn:uuid:999d8ef5-7003-4e58-aaa2-655ffac8ac32	Activity	Processament complet de sample_7	2026-03-31T10:33:00Z	2026-03-31T10:44:49Z	kepler-variant-03	/data/input/	/data/output/sample_7	[object Object],[object Object],[object Object],[object Object]	[object Object],[object Object],[object Object],[object Object]	[object Object]		Processament complet de sample_7	kepler-variant-03	709	OK
8	http://www.w3.org/ns/prov#	urn:uuid:bc0fe4e4-0aa9-44d3-bf57-6aef78314901	Activity	Processament complet de sample_8	2026-03-31T09:27:00Z	2026-03-31T09:29:00Z	atlas-qc-01	/data/input/	/data/output/sample_8	[object Object],[object Object],[object Object],[object Object]	[object Object],[object Object],[object Object],[object Object]	[object Object]	urn:uuid:999d8ef5-7003-4e58-aaa2-655ffac8ac32	Processament complet de sample_8	atlas-qc-01	120	OK
9	http://www.w3.org/ns/prov#	urn:uuid:5ebc76b4-3f1b-422d-9706-c8b84a20461f5	Activity	Processament complet de sample_9	2026-03-31T10:18:00Z	2026-03-31T10:23:48Z	orion-align-02	/data/input/	/data/output/sample_9	[object Object],[object Object],[object Object],[object Object]	[object Object],[object Object],[object Object],[object Object]	[object Object]		Processament complet de sample_9	orion-align-02	348	OK
10	http://www.w3.org/ns/prov#	urn:uuid:2675f279-bc80-4f05-b814-b583c87a33ab	Activity	Processament complet de sample_10	2026-03-31T10:56:00Z	2026-03-31T10:57:43Z	orion-align-02	/data/input/	/data/output/sample_10	[object Object],[object Object],[object Object],[object Object]	[object Object],[object Object],[object Object],[object Object]	[object Object]		Processament complet de sample_10	orion-align-02	103	OK

Showing 1 to 10 of 100 entries Previous 1 2 3 4 5 ... Next

Screen Reader Optimized Ln 286, Col 47 Spaces: 2 UTF-8 LF { } R

Showing 1 to 10 of 100 entries Previous 1 2 3 4 5 ... 10 Next

Activity

Sample: Processament complet de sample_4

Node: atlas-qc-01

User: lab_tech

Duration: 151 s

Size: 2.61 GB

OK

Provenance Graph

Show raw JSON

Agents

seqfu
Type: SoftwareAgent

sha256sum
Type: SoftwareAgent

Pipeline Nextflow fastq_prov
Type: SoftwareAgent

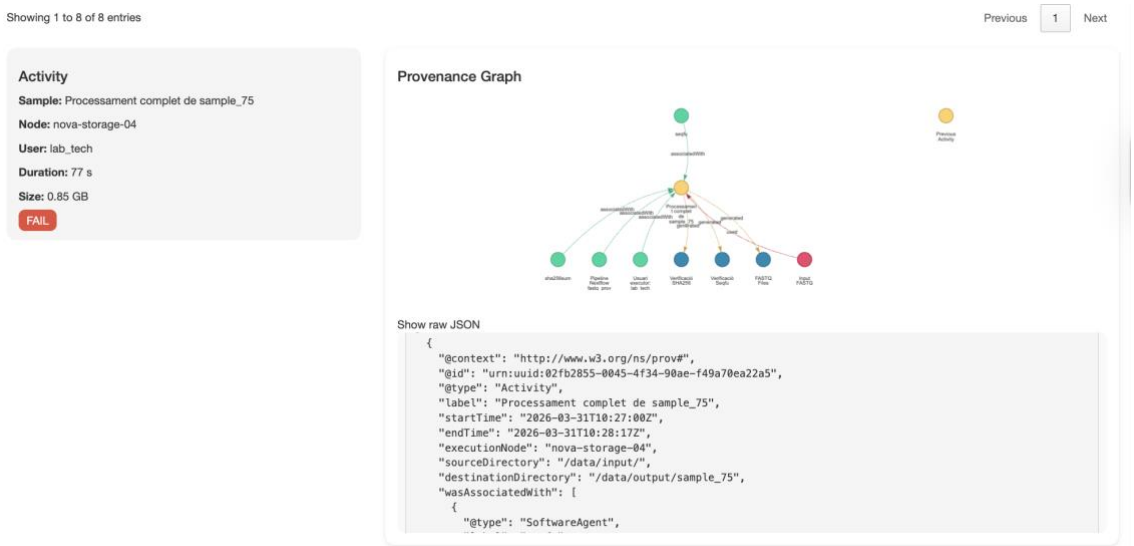
Usuari executor: lab_tech
Type: Person

Generated Entities

Verificació SHA256
sample4_R1.fastq.gz: OK sample4_R2.fastq.gz: OK
sample4_R3.fastq.gz: OK (OK)

Verificació Seqfu
OK (OK)

FASTQ Files
NA (NA)



These images show the detailed view of a single execution in the app. On the left, there is a summary of the activity, including the sample name, execution node, user, duration, data size, and whether the execution was successful or failed.

In the center, the provenance graph visually represents the relationships between the activity, the input data, the generated outputs, and the agents involved, making it easier to understand the workflow.

Below, there is also an option to view the raw JSON of the execution, which shows the complete provenance record in its original structured format for deeper inspection. This is especially useful for debugging, validation, or exporting the data for further analysis outside the app.

References

[1] Shiny - Welcome to Shiny [Internet]. shiny.posit.co. Available from: https://shiny.posit.co/r/getstarted/shiny-basics/lesson1/

[2] Documentación de MongoDB - Página principal [Internet]. Mongodb.com. 2026. Available from: https://www.mongodb.com/es/docs/

[3] PROV-O: The PROV Ontology [Internet]. www.w3.org. Available from: https://www.w3.org/TR/prov-o/

[4] Grolemund G. Shiny - Introduction to interactive documents [Internet]. Shiny. 2014. Available from: https://shiny.posit.co/r/articles/build/interactive-docs/

[5] 43 Dashboards con Shiny – EpiRhandbook en español [Internet]. Epirhandbook.com. 2020 [cited 2026 Apr 27]. Available from: https://epirhandbook.com/es/new_pages/shiny_basics.es.html

[6] Sievert C. Interactive web-based data visualization with R, plotly, and shiny [Internet]. plotly-r.com. Available from: https://plotly-r.com/

[7] Buttons Group checkbox Input Control — checkboxGroupButtons [Internet]. Github.io. 2026 [cited 2026 Apr 27]. Available from: https://dreamrs.github.io/shinyWidgets/reference/checkboxGroupButtons.html

[8] Introduction to dplyr [Internet]. dplyr.tidyverse.org. Available from: https://dplyr.tidyverse.org/articles/dplyr.html

[9] Functional Programming Tools [Internet]. Tidyverse.org. 2019. Available from: https://purrr.tidyverse.org/

[10] Simplification [Internet]. R-Packages. Available from: https://cran.r-project.org/web/packages/jsonlite/vignettes/json-aaquickstart.html

[11] Plotly. Plotly R Graphing Library [Internet]. plotly.com. Available from: https://plotly.com/r/

[12] Using DT in Shiny [Internet]. Github.io. 2024. Available from: https://rstudio.github.io/DT/shiny.html

[13] Introduction [Internet]. datastorm-open.github.io. Available from: https://datastorm-open.github.io/visNetwork/

[14] Shiny [Internet]. Github.io. 2020 [cited 2026 Apr 27]. Available from: https://datastorm-open.github.io/visNetwork/shiny.html